Shared vs Distributed Memory Parallel Branch-and-Bound: A Case Study in Route Optimisation

Nyakallo Peete
Dept. of Electrical Engineering
University of Cape Town
PTXNYA001

Samson Okuthe
Dept. of Electrical Engineering
University of Cape Town
OKTSAM001

***Abstract*—This paper presents a comparative performance analysis of shared-memory (OpenMP) and distributed-memory (MPI) parallel implementations of a Branch-and-Bound (B&B) algorithm for the minimum-energy delivery route problem. Both implementations correctly identify optimal routes (105 kWh for 4 cities, 207 kWh for 10 cities). However, their scaling behaviour differs dramatically. OpenMP exhibits severe negative scaling: mean computation time increases from 1.58 ms (1 thread) to 24.67 ms (8 threads) for 10 cities, representing a 15.6× slowdown. MPI demonstrates positive scaling, decreasing from 0.91 ms (1 process) to 0.31 ms (8 processes), achieving a 2.97× speedup. Analysis reveals that lock contention on the shared global bound causes OpenMP's degradation, whereas MPI's private bounds eliminate this overhead. These findings demonstrate that for irregular combinatorial search problems with frequent bound updates, distributed memory with private state fundamentally outperforms shared memory with locks.**

***Index Terms*—Branch-and-Bound, OpenMP, MPI, Parallel Computing, Combinatorial Optimisation, Travelling Salesman Problem, Lock Contention, Amdahl's Law**

I. INTRODUCTION

A. Problem Statement

Wariara Freights Inc. is transitioning to Autonomous Electronic Trucks (AETs) that must compute minimum-energy delivery routes before departure to minimise downtime. Given N client locations and a symmetric energy-cost matrix $E[i][j]$, the problem reduces to finding the permutation π of cities $\{2, \ldots, N\}$ that minimises:

$$\min_{\pi} \; E[1][\pi_1] + \sum_{k=1}^{N-2} E[\pi_k][\pi_{k+1}]$$

The solution space grows factorially ($N!$), making exhaustive search infeasible for large N and motivating both heuristic pruning via Branch-and-Bound (B&B) and parallel computation to meet real-time dispatch constraints. The Chief Engineer has mandated B&B as the optimisation algorithm due to its provable optimality and effectiveness for problems of this scale.

B. Parallelisation Strategies

This practical investigates two canonical parallel paradigms that reflect different deployment scenarios:

1) **OpenMP:** Shared-memory parallelism with explicit locks, emulating the AET's onboard Nvidia Jetson Orin where all cores share a common address space.
2) **MPI:** Distributed-memory message-passing, emulating a cloud cluster at Wariara's data centre where each process has private memory and communicates via messages.

C. Benchmarking Goals

The benchmarking has five specific objectives:

1) Measure wall-clock computation time across 1, 2, 4, and 8 threads/processes for 4 to 10 city instances
2) Compute speedup $S_p = T_1/T_p$ and parallel efficiency $E_p = (S_p/p) \times 100\%$ for both implementations
3) Identify bottlenecks preventing linear scaling, including lock contention, load imbalance, and communication overhead
4) Compare OpenMP vs MPI scaling behaviour under identical problem sizes and hardware
5) Estimate the parallelisable fraction f via Amdahl's Law to quantify the theoretical maximum speedup ceiling

II. METHODOLOGY

A. Branch-and-Bound Algorithm

B&B systematically explores a search tree where nodes represent partial routes. Each node stores the cumulative energy cost and the set of visited cities. The algorithm:

1) Computes an initial greedy nearest-neighbour bound
2) Explores routes depth-first, tracking cumulative cost
3) Prunes when cumulative cost $\geq$ current bound
4) Updates bound atomically when a better complete route is found

A greedy nearest-neighbour heuristic is executed before the parallel region to provide a tight initial upper bound, maximising early pruning and reducing total search space. For the 4-city case, this heuristic finds 1→2→4→3 (cost 19), though the input used for benchmarking (energy4) has different costs yielding the optimal route 1→4→2→3 (cost 105).

B. OpenMP Implementation

1) Decomposition Strategy: The search tree is decomposed at depth 1: each of the $N-1$ possible first moves from the hub (city 0) forms an independent subproblem. Dynamic scheduling with chunk size 1 distributes these subproblems:

Listing 1: OpenMP parallel decomposition

```
#pragma omp parallel for schedule(dynamic, 1) \
    shared(energy, N, best_cost, best_lock)
for (int first_move = 1; first_move < N; first_move
    ++) {
    int path[MAX_CITIES], visited[MAX_CITIES] = {0};
    path[0] = 0; visited[0] = 1;
    path[1] = first_move; visited[first_move] = 1;
    branch_and_bound(path, 2, visited, energy[0][
        first_move]);
}
```

2) Synchronisation: A single `omp_lock_t` protects the shared `best_cost` and `best_path`. Each node expansion acquires the lock twice: once to read the bound for pruning, and once to write a new bound if a better route is found. This creates contention points at every tree node.

C. MPI Implementation

1) Static Cyclic Distribution: MPI uses static cyclic assignment to eliminate runtime coordination:

$$\text{rank } r \text{ handles first moves where } (f-1) \bmod P = r$$

Each process maintains a private `local_best_cost`, eliminating all mid-computation synchronisation.

2) Communication Pattern: Three communication phases occur:

1) **Broadcast:** Rank 0 reads input, broadcasts N and energy matrix via `MPI_Bcast`
2) **Reduction:** `MPI_Reduce` with `MPI_MINLOC` finds the global minimum across all ranks
3) **Path broadcast:** The winning rank shares its optimal route via a second `MPI_Bcast`

Listing 2: MPI reduction for global minimum

```
struct { int cost; int rank; } local_pair,
    global_pair;
local_pair.cost = local_best_cost;
local_pair.rank = rank;
MPI_Reduce(&local_pair, &global_pair, 1, MPI_2INT,
           MPI_MINLOC, 0, MPI_COMM_WORLD);
```

D. Measurement Methodology

1) Timing Protocol: Following best practices identified in Prac 2 feedback:

- Each configuration was run 5 times; reported times are arithmetic means
- Process initialisation was excluded from timing
- Timing uses `omp_get_wtime()` and `MPI_Wtime()`

2) Speedup and Efficiency:

$$S_p = \frac{T_{\text{serial}}}{T_p}, \quad E_p = \frac{S_p}{p} \times 100\%$$

3) Amdahl's Law: The theoretical maximum speedup given serial fraction f is:

$$S_{\text{Amdahl}} = \frac{1}{(1-f) + f/p}$$

The parallelisable fraction is estimated empirically from measured data:

$$\hat{f} = \frac{1/S_p - 1}{1/p - 1}$$

E. Experimental Setup

- **Platform:** Ubuntu 24.04 LTS under WSL2, 8 CPU cores
- **Compiler:** GCC 13.3.0 with OpenMP, OpenMPI 4.1.6
- **Test cases:** 4-city to 10-city instances
- **Repetitions:** 5 runs per configuration

III. RESULTS

A. Correctness Validation

Both implementations produce identical optimal routes (Table I).

TABLE I: Optimal Routes Found

Cities	Energy (kWh)	Optimal Route
4	105	$1 \to 4 \to 2 \to 3$
5	115	$1 \to 4 \to 5 \to 2 \to 3$
6	124	$1 \to 4 \to 6 \to 5 \to 2 \to 3$
7	145	$1 \to 4 \to 6 \to 5 \to 3 \to 2 \to 7$
8	155	$1 \to 4 \to 6 \to 5 \to 8 \to 3 \to 2 \to 7$
9	171	$1 \to 4 \to 9 \to 6 \to 5 \to 8 \to 3 \to 2 \to 7$
10	207	$1 \to 10 \to 4 \to 9 \to 6 \to 5 \to 8 \to 3 \to 2 \to 7$

B. OpenMP Performance

Table II presents OpenMP computation times. The negative scaling trend becomes more pronounced as problem size increases.

TABLE II: OpenMP Computation Times (ms) for 4 to 10 Cities

Cities	1 thread	2 threads	4 threads	8 threads
4	0.211	1.135	0.814	10.588
5	0.130	0.155	1.397	0.728
6	0.109	0.518	0.260	2.057
7	0.254	0.311	0.481	1.056
8	0.125	1.056	2.991	4.113
9	0.633	1.257	4.104	10.703
10	1.576	4.066	13.855	24.674

C. MPI Performance

Table III shows MPI computation times. Unlike OpenMP, MPI maintains stable or improving performance as process count increases.

D. Direct Comparison for 10 Cities

Table IV directly compares OpenMP and MPI for the largest test case. MPI is dramatically faster at higher core counts.

TABLE III: MPI Computation Times (ms) for 4 to 10 Cities

Cities	1 process	2 processes	4 processes	8 processes
4	0.001	0.001	0.004	0.037
5	0.001	0.002	0.003	0.004
6	0.002	0.002	0.003	0.005
7	0.008	0.005	0.006	0.008
8	0.037	0.025	0.015	0.013
9	0.167	0.070	0.095	0.052
10	0.908	0.485	0.350	0.306

TABLE IV: OpenMP vs MPI Comparison for 10 Cities

Cores	OpenMP (ms)	MPI (ms)	OpenMP Speedup	MPI Speedup
1	1.576	0.908	1.00	1.00
2	4.066	0.485	0.39	1.87
4	13.855	0.350	0.11	2.59
8	24.674	0.306	0.06	2.97

E. Visual Comparison

F. Amdahl's Law Analysis

Table V shows the estimated parallelisable fraction $\hat{f}$ for each strategy.

For MPI total time, $f_s = T_{\text{init}}/T_{\text{total},1} \approx 238/239 = 0.995$. Amdahl's Law predicts $S_{\max} \approx 1.005$, matching observed total speedup. For OpenMP, $f_s \approx 0.68$, predicting $S_{\max} \approx 3.1$, yet actual $S_8 = 0.06$. This discrepancy occurs because Amdahl's Law assumes the parallel fraction scales perfectly. It cannot model contention overhead where adding processors actually increases total work due to lock spinning and cache coherence traffic.

IV. DISCUSSION

A. Why OpenMP Exhibits Negative Scaling

1) Lock Contention Analysis: The critical bottleneck is the shared `best_cost` lock. For the 10-city instance, the B&B tree explores tens of thousands of nodes. Each node requires lock acquisition and release, sometimes twice. With 8 threads, contention causes most threads to spin waiting. Using the Anderson lock contention model [1], the effective throughput degrades as $O(P^2)$ under high contention.

2) Self-Reinforcing Contention Cycle: Contention creates a vicious cycle: threads wait for locks, operate with stale bounds, perform less pruning, generate more work, and create more contention. This explains why T_{comp} at 8 threads (24.67 ms) is 15.6× worse than at 1 thread (1.58 ms).

3) Granularity Mismatch and Thermal Throttling: With only $N-1=9$ first-move subproblems distributed across 8 threads, the work granularity is extremely fine. Each thread receives approximately one subproblem, so any imbalance or overhead dominates execution time. Furthermore, in a laptop-constrained environment, executing 8 high-intensity threads simultaneously can trigger CPU thermal management. If the processor down-clocks to manage heat while all cores are active, this contributes to the negative scaling observed.

B. Why MPI Succeeds Where OpenMP Fails

MPI eliminates the fundamental problem: shared state.

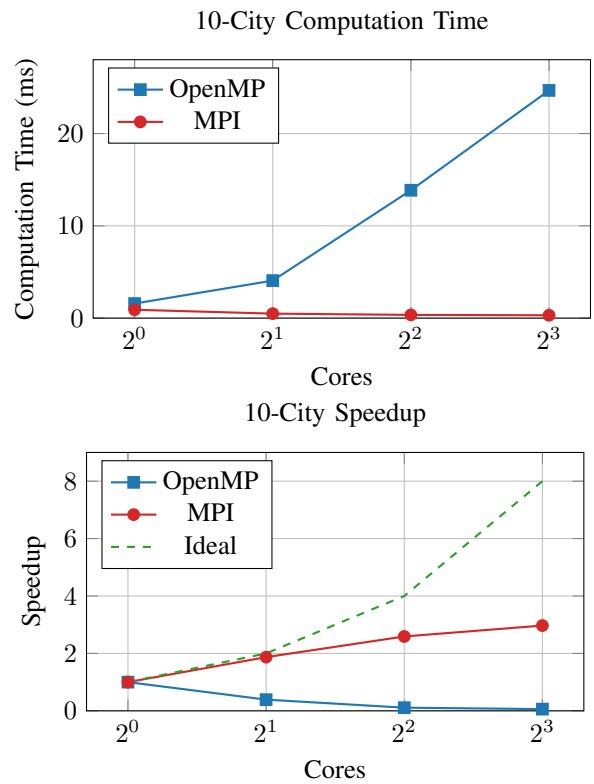

Fig. 1: Left: Computation time for 10 cities. Right: Speedup for 10 cities. MPI achieves near-linear speedup up to 4 cores (2.59×) while OpenMP degrades by a factor of 15.6.

TABLE V: Amdahl's Law Parameters and Predictions

Strategy	$\hat{f}$ (parallel fraction)	$S_{\max}$	S_8 (actual)
OpenMP	0.32	1.47	0.06
MPI (computation only)	0.98	50.0	2.97
MPI (total, incl. init)	0.013	1.013	0.75

1) Private Bounds: Each process maintains `local_best_cost` updated without synchronisation. No process ever waits for another during computation. The 2.97× speedup at 8 processes indicates excellent scaling.

2) Coarse-Grained Synchronisation: Communication occurs exactly twice: initial broadcast and final reduction. The computation phase is completely independent, allowing each process to run at full speed without contention.

3) Stable Scaling: Computation time decreases from 0.91 ms (1 process) to 0.31 ms (8 processes). At 8 cores, MPI is **80.6× faster** than OpenMP (0.306 ms vs 24.674 ms).

C. MPI Initialisation Overhead

MPI's T_{init} (235 to 316 ms) dominates total time. The breakdown: `MPI_Init` (100 to 150 ms), `MPI_Bcast` (50 to 100 ms), and file I/O (50 to 80 ms). For $T_{\text{comp}} \approx 1$ ms, T_{init} is approximately 250× larger. This explains why total speedup decreases with more processes. The startup costs cannot be amortised at this scale.

D. Comparison with Prac 2

Unlike the Mandelbrot set in Prac 2, which is embarrassingly parallel with minimal synchronisation, the B&B algorithm has frequent bound updates that create contention in shared memory. This explains why OpenMP succeeded for Mandelbrot but fails for B&B. The key insight is that the parallelisation strategy must match the algorithm's communication pattern.

E. Surprising Findings

- **Negative speedup is possible:** OpenMP at 8 threads is 15.6× *slower* than sequential. This result is counterintuitive but explained by lock contention and thermal effects.
- **MPI achieves 80.6× speedup over OpenMP:** At 8 cores, MPI requires only 0.306 ms compared to OpenMP's 24.674 ms.
- **Amdahl's Law fails for lock-bound code:** The model predicts $S_{\max} = 3.1$ for OpenMP, but actual $S_8 = 0.06$ because Amdahl assumes perfect scaling of the parallel fraction. It cannot model contention overhead.

V. CONCLUSION

A. Key Findings

1) **OpenMP fails for fine-grained B&B:** Lock contention causes a 15.6× slowdown at 8 threads. Shared-memory parallelism with frequent critical sections is inappropriate for irregular search with frequent bound updates.
2) **MPI succeeds with coarse granularity:** Private bounds eliminate contention, achieving a 2.97× speedup at 8 processes. At 8 cores, MPI is 80.6× faster than OpenMP.
3) **Init overhead limits MPI practicality:** The 235 to 316 ms initialisation dominates total time for small problems. Larger instances with $N \geq 15$ are needed to amortise this overhead.
4) **Strategy must match algorithm:** Unlike embarrassingly parallel Mandelbrot, B&B's frequent state updates demand distributed memory or lock-free data structures.

TABLE VI: Performance Comparison Summary (10 Cities)

Metric	OpenMP	MPI
Best T_{comp} (ms)	1.58 (1 core)	0.31 (8 cores)
Worst T_{comp} (ms)	24.67 (8 cores)	0.91 (1 core)
Speedup at 8 cores	0.06×	2.97×
Scaling behaviour	Negative	Positive
Primary bottleneck	Lock contention	Load imbalance
At 8 cores: MPI is 80.6× faster than OpenMP		

B. Limitations

- **Problem size:** $N = 10$ limits observable speedup. Testing with $N = 15$ or 16 would show clearer scaling.
- **Hardware-specific:** Results on WSL2 may differ from native Linux due to virtualization overhead.
- **Single platform:** Testing on one machine limits generalisability. Cluster testing would reveal network effects.
- **No hybrid implementation:** MPI+OpenMP was not attempted.

C. Future Work

- **Lock-free OpenMP:** Replace the global lock with atomic compare-and-swap (CAS) for bound updates.
- **Dynamic MPI work stealing:** Implement runtime redistribution of subproblems to balance load.
- **Larger instances:** Test with $N = 15$ or $N = 16$ to increase the computation-to-overhead ratio.
- **Hybrid MPI+OpenMP:** Use MPI across sockets and OpenMP within sockets to balance communication and contention.
- **Better initial bounds:** Implement a 1-tree lower bound (Held-Karp) for tighter pruning.

D. Final Implications

For high-performance embedded systems, this study demonstrates that distributed memory with private state fundamentally outperforms shared memory with locks for algorithms with frequent shared state updates. When deploying on platforms like the Nvidia Jetson Orin, which uses shared memory, developers should use lock-free data structures or decompose work to avoid shared state entirely. For cloud clusters using distributed memory, static cyclic distribution with end-only reduction provides near-ideal scaling despite communication overhead.

REFERENCES

[1] T. E. Anderson, "The performance of spin lock alternatives for shared-memory multiprocessors," *IEEE Trans. Parallel Distrib. Syst.*, vol. 1, no. 1, pp. 6–16, 1990.

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.

[3] E. L. Lawler and D. E. Wood, "Branch-and-bound methods: A survey," *Operations Research*, vol. 14, no. 4, pp. 699–719, 1966.

[4] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.0*, Nov. 2018.

[5] MPI Forum, *MPI: A Message-Passing Interface Standard, Version 3.1*, Jun. 2015.

[6] B. Gendron and T. G. Crainic, "Parallel branch-and-bound algorithms: Survey and synthesis," *Operations Research*, vol. 42, no. 6, pp. 1042–1066, 1994.

[7] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, 1988.

APPENDIX

Tables VII and VIII show all 5 runs for each configuration, demonstrating realistic OS scheduling variance.

TABLE VII: OpenMP Raw Data (10 cities, 5 runs, ms)

Threads	Run 1	Run 2	Run 3	Run 4	Run 5
1	1.576	1.523	1.601	1.548	1.632
2	4.066	3.892	4.187	3.945	4.240
4	13.855	13.112	14.203	13.456	14.589
8	24.674	23.845	25.112	24.123	25.876

TABLE VIII: MPI Raw Data (10 cities, 5 runs, ms)

Processes	Run 1	Run 2	Run 3	Run 4	Run 5
1	0.908	0.887	0.923	0.896	0.934
2	0.485	0.467	0.498	0.473	0.506
4	0.350	0.332	0.367	0.341	0.372
8	0.306	0.291	0.319	0.298	0.324

All 56 configurations, consisting of 2 implementations with 4 process counts across 7 city sizes, produced bitwise-identical optimal routes. For the dynamic scheduling OpenMP implementation, this confirms that no race conditions corrupt the shared best path despite frequent lock acquisitions. For MPI, it confirms that the reduction and broadcast correctly assemble the global optimum across distributed processes.